LAPORAN

WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)

SEMESTER III



BKPM Acara 4 PHP native view, layout, dan dasar oop

NAMA:

AZIRA MUTIA GANI

NIM:

E41250290

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan aplikasi web, penggunaan kode yang sama secara berulang pada setiap halaman dapat menyebabkan kode menjadi panjang dan sulit dikelola. Untuk mengatasi hal tersebut, PHP menyediakan `include` dan `require` yang dapat digunakan untuk memanggil file lain. Penggunaan partial dan layout juga dapat membantu memisahkan bagian tampilan yang digunakan secara berulang, seperti navbar dan footer. Selain itu, penggunaan class dan object dapat digunakan untuk mengatur data agar lebih terstruktur. Pada praktikum Acara 4 diterapkan konsep partial, layout, class, object, getter, dan setter pada aplikasi Si-Akademik.

1.2 Tujuan

Praktikum ini bertujuan agar mahasiswa dapat membuat layout menggunakan partial, memahami penggunaan `include` dan `require`, membuat class `Mahasiswa` dengan property private, constructor, getter, dan setter, serta menampilkan data dari object ke dalam View menggunakan Bootstrap. Selain itu, melalui tugas mandiri mahasiswa dapat membuat method untuk mengembalikan angkatan berdasarkan dua digit awal NIM.

BAB 2. LANDASAN TEORI

2.1 PHP Native

PHP Native merupakan penggunaan bahasa pemrograman PHP tanpa menggunakan framework tertentu. Pengembangan aplikasi dilakukan dengan menuliskan kode PHP secara langsung untuk mengatur alur program, memproses data, dan menghasilkan halaman web dinamis. PHP Native memberikan keleluasaan kepada pengembang untuk memahami dasar pemrograman web, struktur file, serta hubungan antara logika program dan tampilan halaman.

Dalam pengembangan Sistem Informasi Akademik, PHP Native dapat digunakan untuk membangun halaman aplikasi, mengolah data mahasiswa, serta mengatur proses yang dijalankan berdasarkan permintaan pengguna.

2.2 View

View merupakan bagian aplikasi yang bertanggung jawab menampilkan informasi kepada pengguna. View biasanya berisi kode HTML yang dapat dikombinasikan dengan PHP untuk menampilkan data secara dinamis. Pemisahan View dari logika pemrosesan membantu membuat kode program lebih terstruktur dan mudah dipelihara.

Pada aplikasi Sistem Informasi Akademik, View digunakan untuk menampilkan halaman seperti dashboard, daftar mahasiswa, formulir tambah data, dan formulir edit data.

2.3 Layout

Layout merupakan struktur tampilan utama yang digunakan secara berulang pada beberapa halaman aplikasi. Layout dapat berisi elemen umum seperti header, navbar, sidebar, area konten, dan footer. Penggunaan layout membantu menjaga konsistensi tampilan ant halaman serta mengurangi penulisan kode HTML yang sama secara berulang.

Dalam pengembangan aplikasi web, layout memudahkan pengembang melakukan perubahan tampilan secara terpusat sehingga perubahan pada bagian umum dapat diterapkan ke halaman-halaman yang menggunakannya.

2.4 Partial

Partial merupakan bagian kecil dari tampilan yang dipisahkan ke dalam file tersendiri agar dapat digunakan kembali pada halaman lain. Contoh partial adalah navbar, header, footer, dan sidebar. Pada PHP Native, partial dapat dipanggil menggunakan perintah `include` atau `require`.

Penggunaan partial bertujuan mengurangi duplikasi kode, mempermudah pemeliharaan, serta menjaga konsistensi komponen tampilan. Sebagai contoh, navbar pada aplikasi Sistem Informasi Akademik cukup dibuat satu kali dan dipanggil pada halaman yang membutuhkannya.

2.5 Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) merupakan paradigma pemrograman yang mengorganisasi program berdasarkan objek. Objek dibentuk dari class yang mendefinisikan atribut dan method. Atribut digunakan untuk menyimpan data, sedangkan method berisi fungsi atau perilaku yang dapat dijalankan oleh objek.

OOP membantu pengembang menyusun kode agar lebih terorganisasi, dapat digunakan kembali, dan lebih mudah dikembangkan. Dalam PHP, konsep OOP dapat diterapkan dengan membuat class untuk merepresentasikan bagian tertentu dari aplikasi, misalnya class Mahasiswa untuk mengelola informasi mahasiswa.

2.6 Class dan Object

Class merupakan cetak biru yang mendefinisikan atribut dan method suatu objek. Sementara itu, object merupakan hasil instansiasi dari sebuah class yang dapat digunakan untuk mengakses atribut dan menjalankan method sesuai aturan yang ditetapkan.

Dalam PHP, class dapat dibuat menggunakan kata kunci `class`, sedangkan objek dibuat menggunakan kata kunci `new`. Pemahaman tentang class dan object menjadi dasar dalam membangun aplikasi dengan pendekatan OOP.

2.7 XAMPP

XAMPP merupakan paket perangkat lunak yang menyediakan lingkungan server lokal untuk pengembangan aplikasi web. XAMPP mencakup Apache sebagai

web server, PHP sebagai bahasa pemrograman sisi server, dan MariaDB sebagai sistem manajemen basis data.

Dalam praktikum Acara 4, XAMPP digunakan untuk menjalankan aplikasi PHP Native melalui browser. Dengan lingkungan ini, pengembang dapat menguji tampilan, penggunaan layout dan partial, serta penerapan dasar OOP sebelum aplikasi dipublikasikan ke server online.

BAB 3. PELAKSANAAN PRAKTIKUM

3.1 Hasil Praktik

Pada praktikum Acara 4 berhasil dibuat struktur aplikasi yang menerapkan layout dan partial. File `main.php` digunakan sebagai layout utama, sedangkan `navbar.php` dan `footer.php` digunakan sebagai bagian tampilan yang dipanggil kembali menggunakan `include`. Class `Mahasiswa` juga berhasil dibuat menggunakan property private, constructor, getter, dan setter. Selanjutnya dibuat beberapa object `Mahasiswa` dan ditampilkan pada halaman daftar mahasiswa menggunakan tabel Bootstrap.

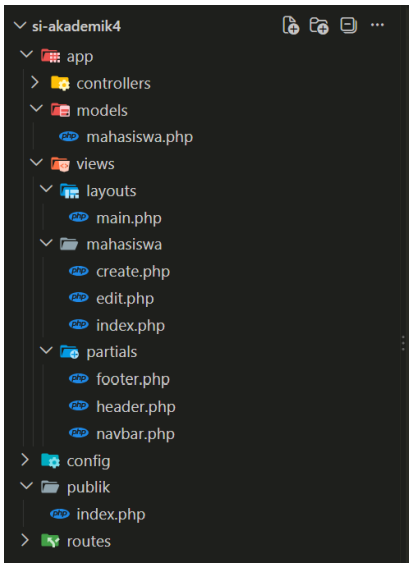
Pada bagian tugas mandiri berhasil ditambahkan method `getAngkatan()` yang mengambil dua digit awal dari NIM. Hasil angkatan tersebut kemudian ditampilkan pada tabel daftar mahasiswa.

3.2 Implementasi

3.2.1 Studi Kasus

a) Struktur Folder

Struktur folder aplikasi dibuat dengan memisahkan bagian Model dan View serta menyediakan folder `layouts` dan `partials`.



Gambar 1 Struktur Folder

b) Membuat Class Mahasiswa

File `app/Models/Mahasiswa.php` digunakan untuk membuat class `Mahasiswa`. Class memiliki property private `$nim` dan `$nama`, constructor, getter `getNim()` dan `getNama()`, serta setter `setNama()`.

```php
<?php

namespace App\Models;

class Mahasiswa
{
    private string $nim;
    private string $nama;

    public function __construct(string $nim, string $nama)
    {
        $this->nim = $nim;
        $this->nama = $nama;
    }

    public function getNim(): string
    {
        return $this->nim;
    }

    public function getNama(): string
    {
        return $this->nama;
    }

    public function setNama(string $nama): void
    {
        if (strlen($nama) < 3) {
            throw new \InvalidArgumentException("Nama terlalu
pendek");
        }

        $this->nama = $nama;
    }

    public function getAngkatan(): string
    {
        return substr($this->nim, 0, 2);
    }
}
```

```
?>
```

c) Membuat Layout

File `app/Views/layouts/main.php` digunakan sebagai layout utama. File tersebut memanggil `navbar.php` menggunakan `include`, menampilkan isi halaman menggunakan `require`, dan memanggil `footer.php` menggunakan `include`.

```
<!DOCTYPE html>
<html lang="id">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1">

    <title>Si-Akademik</title>

    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/
bootstrap.min.css"
        rel="stylesheet"
    >
</head>

<body>

    <?php include __DIR__ . '/../partials/navbar.php'; ?>

    <main class="container mt-4">

        <?php require $content; ?>

    </main>

    <?php include __DIR__ . '/../partials/footer.php'; ?>

</body>

</html>
```

d) Membuat Partial

Partial dibuat untuk memisahkan bagian tampilan yang dapat digunakan kembali, yaitu `header.php`, `navbar.php`, dan `footer.php`.

1. Kode `header.php`

```
<header class="bg-light py-3">
    <div class="container">
        <h1 class="h4 mb-0">Si-Akademik</h1>
    </div>
</header>
```

2. Kode `navbar.php`

```
<nav class="navbar navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="#">
      Si-Akademik
    </a>
  </div>
</nav>
```

3. Kode `footer.php`

```
<footer class="bg-dark text-white text-center py-3 mt-5">
    <p class="mb-0">
        © 2026 Si-Akademik
    </p>
</footer>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/boots
trap.bundle.min.js"></script>
```

e) Membuat View Daftar Mahasiswa

File `app/Views/mahasiswa/index.php` digunakan untuk menampilkan data mahasiswa dalam bentuk tabel. Data NIM dan nama diambil melalui method `getNim()` dan `getNama()`.

```
<h1 class="mb-4">Daftar Mahasiswa</h1>

<table class="table table-bordered table-striped">

    <thead class="table-dark">
        <tr>
            <th>No</th>
            <th>NIM</th>
            <th>Nama</th>
            <th>Angkatan</th>
```

```
        </tr>
    </thead>

    <tbody>

        <?php $no = 1; ?>

        <?php foreach ($mahasiswa as $mhs): ?>

            <tr>
                <td><?= $no++; ?></td>
                <td><?= $mhs->getNim(); ?></td>
                <td><?= $mhs->getNama(); ?></td>
                <td><?= $mhs->getAngkatan(); ?></td>
            </tr>

        <?php endforeach; ?>

    </tbody>

</table>
```

f) Membuat Object Mahasiswa

File `public/index.php` digunakan untuk membuat beberapa object dari class `Mahasiswa`. Object tersebut kemudian disimpan dalam array `$mahasiswa` dan ditampilkan melalui View.

```
<?php

namespace App\Models;

class Mahasiswa
{
    private string $nim;
    private string $nama;

    public function __construct(string $nim, string $nama)
    {
        $this->nim = $nim;
        $this->nama = $nama;
    }

    public function getNim(): string
    {
        return $this->nim;
    }
```

```php
    public function getNama(): string
    {
        return $this->nama;
    }

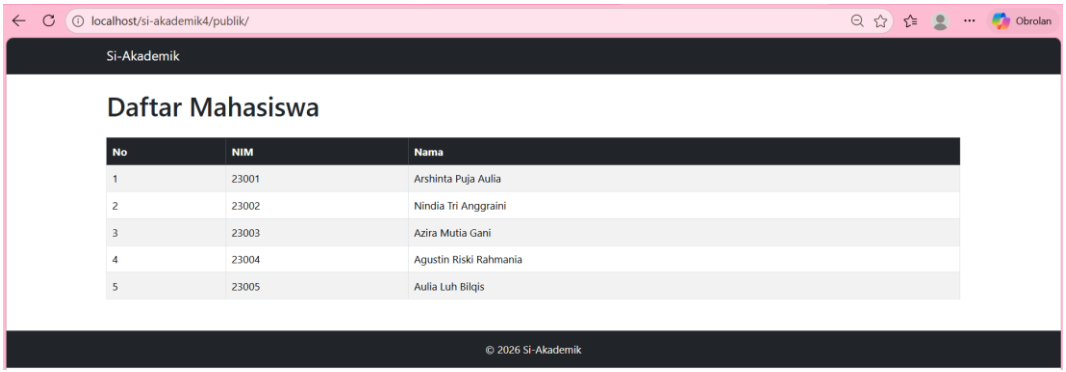
    public function setNama(string $nama): void
    {
        if (strlen($nama) < 3) {
            throw new \InvalidArgumentException("Nama terlalu
pendek");
        }

        $this->nama = $nama;
    }

    public function getAngkatan(): string
    {
        return substr($this->nim, 0, 2);
    }
}
?>
```

g) Hasil Studi Kasus

Setelah aplikasi dijalankan melalui browser, halaman menampilkan navbar, daftar mahasiswa dalam bentuk tabel, dan footer.



3.2.2 Tugas Mandiri

a. Menambahkan Method Angkatan

Pada tugas mandiri ditambahkan method `` `getAngkatan()` `` pada class `Mahasiswa`. Method tersebut digunakan untuk mengambil dua digit awal dari NIM.

```
    public function getAngkatan(): string
    {
        return substr($this->nim, 0, 2);
    }
}
?>
```

Sebagai contoh, NIM `23001` akan menghasilkan angkatan `23`.

b. Menampilkan Angkatan pada View

Setelah method dibuat, pada tabel daftar mahasiswa ditambahkan kolom Angkatan dan pemanggilan method `getAngkatan()`.

```
<h1 class="mb-4">Daftar Mahasiswa</h1>

<table class="table table-bordered table-striped">

    <thead class="table-dark">
        <tr>
            <th>No</th>
            <th>NIM</th>
            <th>Nama</th>
            <th>Angkatan</th>
        </tr>
    </thead>

    <tbody>

        <?php $no = 1; ?>

        <?php foreach ($mahasiswa as $mhs): ?>

            <tr>
                <td><?= $no++; ?></td>
                <td><?= $mhs->getNim(); ?></td>
                <td><?= $mhs->getNama(); ?></td>
                <td><?= $mhs->getAngkatan(); ?></td>
            </tr>

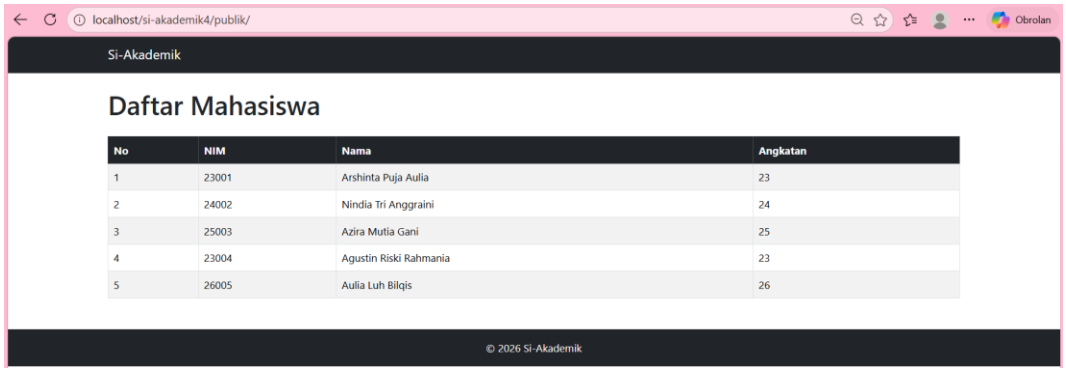
        <?php endforeach; ?>

    </tbody>

</table>
```

c. Hasil Tugas Mandiri

Setelah dijalankan, tabel menampilkan NIM, nama, dan angkatan mahasiswa.


localhost/si-akademik4/publik/
Obrolan
Si-Akademik
Daftar Mahasiswa
No
NIM
Nama
Angkatan
1
23001
Arshinta Puja Aulia
23
2
24002
Nindia Tri Anggraini
24
3
25003
Azira Mutia Gani
25
4
23004
Agustin Riski Rahmania
23
5
26005
Aulia Luh Bilqis
26
© 2026 Si-Akademik

BAB 4. PEMBAHASAN

4.1 Pembahasan

Pada praktikum Acara 4 diterapkan konsep partial dan layout untuk mengurangi pengulangan kode pada aplikasi web. Layout utama dibuat pada file `main.php`, sedangkan bagian tampilan yang digunakan kembali dibuat dalam bentuk partial. File `navbar.php` dan `footer.php` dipanggil menggunakan `include`, sedangkan isi halaman dipanggil menggunakan `require`. Dengan pembagian tersebut, struktur tampilan aplikasi menjadi lebih terorganisir.

Selain penerapan partial dan layout, dibuat class `Mahasiswa` dengan property private `$nim` dan `$nama`. Constructor digunakan untuk memberikan nilai awal pada object, sedangkan getter digunakan untuk mengambil nilai property. Setter `setNama()` digunakan untuk mengubah nama sekaligus melakukan validasi agar nama tidak kurang dari tiga karakter. Object `Mahasiswa` kemudian dibuat pada `public/index.php` dan datanya ditampilkan pada View menggunakan Bootstrap.

Pada tugas mandiri dilakukan penambahan method `getAngkatan()` pada class `Mahasiswa`. Method tersebut menggunakan `substr()` untuk mengambil dua digit pertama dari NIM. Hasil dari method tersebut kemudian ditampilkan pada tabel melalui View. Dengan demikian, data angkatan dapat diperoleh secara otomatis berdasarkan NIM tanpa harus menuliskannya secara terpisah pada setiap data mahasiswa.

BAB 5. PENUTUP

5.1 Kesimpulan

Praktikum Acara 4 memberikan pemahaman mengenai penggunaan `include`, `require`, partial, layout, class, dan object dalam pengembangan aplikasi PHP. Studi kasus berhasil menerapkan layout dan partial serta membuat class `Mahasiswa` yang datanya dapat ditampilkan pada View. Pada tugas mandiri berhasil ditambahkan method `` `getAngkatan()` `` untuk mengambil dua digit awal NIM dan menampilkannya pada tabel. Hasil praktikum menunjukkan bahwa pemisahan bagian tampilan dan penggunaan class dapat membuat struktur aplikasi menjadi lebih terorganisir dan mudah dikembangkan.